Test Paper- A

Table Schema

Astronauts:

astronaut_id:, Auto Increment– Unique identifier for each astronaut.
astronaut_name: Full name of the astronaut.
age: Current age of the astronaut.
nationality: Nationality of the astronaut.
total_space_missions: The number of space missions they have participated in.

Missions:

mission_id: Auto Increment– Unique identifier for each space mission.
mission_name: Name of the space mission.
launch_date: The date of the mission's launch.
duration_days: Total duration of the mission in days.
mission_type: The type of mission (e.g., research, exploration).

Spacecrafts:

spacecraft_id: Auto Increment– Unique identifier for each spacecraft.
spacecraft_name: Name of the spacecraft.
capacity: Number of astronauts the spacecraft can carry.
manufacturer: The company or country that manufactured the spacecraft.

Participation:

astronaut_id: Foreign Key – Links to the Astronauts table.
mission_id: Foreign Key – Links to the Missions table.
spacecraft_id: Foreign Key – Links to the Spacecrafts table.
role: The role of the astronaut in the mission (e.g., commander, engineer).

Consider above table schema and write following queries:

1.	Retrieve the distinct mission names where the mission lasted more than 30 days.
2.	Retrieve the top 3 astronauts who participated in the most missions, ensuring no duplicates.
3.	Insert a new space mission called "Jupiter Exploration" that is scheduled to launch on '2024-11-01', lasting 365 days, and classified as an exploration mission.
4.	Update the total space missions count for astronaut with ID = 5, increasing it by 1.
5.	Delete participation record for astronaut ID 3 in mission ID 2.
6.	Add a new column experience_level (VARCHAR(50)) to the Astronauts table to store the astronaut's experience level.
7.	Clear all the data from the Participation table.(Truncate)
8.	Retrieve all mission names where the mission type contains 'exploration'.
9.	Retrieve all missions that contain the word "Mars" and lasted more than 100 days.
10.	Retrieve the square root of the total number of missions for astronaut ID 102
11.	Retrieve the first 3 characters of each astronaut's name.

12.	Retrieve the astronauts who participated in missions launched in the current year.
13.	Count the number of astronauts from each nationality who have participated in more than 2 space missions.
14.	Retrieve the total number of missions and the average mission duration for each mission type, but only include mission types that have been involved in more than 3 missions.
15.	Find the number of missions commanded by astronauts for each nationality where more than 5 missions were commanded
16.	Retrieve the name of the spacecraft used in the mission "Apollo 11" (Use sub Query)
17.	Create a view that shows all active missions (those that launched after 2020).
18.	List all astronauts and their respective spacecraft for each mission they participated in.
19.	Retrieve the names of astronauts who participated in missions using spacecrafts manufactured by "SpaceX", along with the names of those missions and the duration of each mission. Include only astronauts who have participated in more than 2 missions.
20.	Retrieve the names of astronauts, the names of missions they participated in, the names of spacecraft used in those missions, and the manufacturers of those spacecraft, for missions where the mission duration is greater than the average duration of all missions conducted by astronauts from the "USA".

Test Paper- B

Table Schema

Team Table:

- **team_id**: stores the unique ID for each team.
- **team_name**: stores the name of the cricket team.
- **team_coach**: stores the name of the team's coach.
- **team_wins:** stores the total number of matches the team has won.
- **team_total_matches**: stores the total number of matches the team has played.
- **home_stadium_id:** stores the ID of the team's home stadium (references **stadium_id** in the Stadium table).

Player Table:

- **player_id**: stores the unique ID for each player.
- **player_first_name**: stores the player's first name.
- **player_last_name**: stores the player's last name.
- **team_id**: stores the ID of the team the player belongs to (references the **team_id** in the Team table).
- **player_role**: stores the player's role, such as Batsman, Bowler, or All-rounder.
- **player_jersey_number**: stores the player's jersey number.
- **player_matches_played:** stores the total number of matches the player has played.

Stadium Table:

- **stadium_id:** stores the unique ID for each stadium.
- **stadium_name**: stores the name of the stadium.
- **stadium_city**: stores the city where the stadium is located.
- **stadium_capacity**: stores the seating capacity of the stadium.

Consider above table schema and write following queries:

1.	Retrieve Unique Roles of Players.
2.	Calculate the winning percentage of each team.
3.	Insert a new record to Stadium Table. (2, Wankhede Stadium, Mumbai,33000)
4.	Update Team Coach Name of Team RCB to Ashish Nehra.
5.	Delete All the Records of Player Shikhar Dhawan.
6.	Change the size of stadium_name column from VARCHAR (100) to VARCHAR (15).
7.	Remove Player Table.
8.	Display Top 30 Players Whose First Name Starts with Vowel.
9.	Display City Whose Stadium Name does not Ends with 'M'.
10.	Generate Random Number between 0 to 100.
11.	Display 4th to 9th Character of 'Virat Kohli'.
12.	Display The Day of week on 01-01-2005.
13.	Display City Wise Maximum Stadium Capacity.
14.	Display City Whose Average Stadium Capacity is Above 20000.
15.	Display All Players Full Name, Jersey No and Role Who is Playing for Mumbai Indians. (Using Sub query).
16.	Display Team Name Having Home Stadium Capacity Over 50000. (Using Sub query).

17.	Create a View Players_Above_100_Matches of Players Who have Played More than 100 Matches.
18.	Get the Player name, Team name, and their Jersey number who have played Between than 50 to 100 matches.
19.	Produce Output Like: <PlayerFirstName> Plays For <TeamName> and Has Played <PlayerMatches> matches. (In single column)
20.	Display Stadium Capacity of Team CSK.

Test Paper- C

Table Schema

Table1:**MovieDetails**

Column Name	Datatype	Discription
MovieID	INT (Primary Key)	Unique identifier for each movie.
Title	VARCHAR(100)	Title of the movie.
Genre	VARCHAR(100)	Genre(s) of the movie, such as Drama, Crime, Action.
Director	VARCHAR(100)	Name(s) of the director(s) of the movie.
ReleaseYear	INT	Year the movie was released.

Table2: **MovieFinancials**

Column Name	Datatype	Discription
FinancialID	INT (Primary Key)	Unique identifier for each financial record.
BudgetUSD	DECIMAL	Production budget of the movie in USD.
BoxOfficeUSD	DECIMAL	Box office revenue of the movie in USD.
MovieID	INT (Foreign Key)	References the unique identifier in the MovieDetails table.

Table3: **MovieRatingsDuration**

Column Name	Datatype	Discription
RatingID	INT (Primary Key)	Unique identifier for each rating and duration record.
DurationMin	INT	Duration of the movie in minutes.
Rating	DECIMAL	Average rating of the movie, typically on a scale from 1 to 10.
Language	VARCHAR(100)	The primary language of the movie.
Country	VARCHAR(100)	The country where the movie was produced.
MovieID	INT (Foreign Key)	References the unique identifier in the MovieDetails table.

Consider above table schema and write following queries:

1.	Retrive first five distinct movies along with their title from MovieDetails table.
2.	Display the total of the BudgetUSD and BoxOfficeUSD assign the name TotalUSD from MovieFinancials.
3.	Insert the new row with this data (11,The Incredible Hulk, Action, Louis Leterrier,2008) in MovieDetails table.
4.	Set the value of the genre to 'Action' of 'Avengers:Endgame' movie from MovieDetails table.
5.	Delete the records with duration of 181 minutes from MovieRatingsDuration table.
6.	Add a new column 'Producer' into the MovieDetails table.
7.	Delete records of MovieFinancials table without removing its table structure.
8.	Retrive all the movies from MovieDetails table with title starting with 'The'.
9.	Retrive name of directors includes 'son' from MovieDetails table.
10.	Convert and display title of all movies in uppercase.

11.	Display the highest rating from the MovieRatingsDuration table.
12.	Calculate the years between current year and movies release year.
13.	Find the languages in which movies have an average rating of greater than 8.0. Display the language and the average rating.
14.	Retrieve the minimum, maximum, and average movie duration for each language in the MovieRatingsDuration table, but display only those languages where the average rating is greater than 7.5.
15.	Find the titles of movies whose budget is higher than the average budget of all movies.(Do not use JOINS)
16.	Find the titles of movies that have a box office revenue greater than the average box office revenue of all movies.
17.	Create a view with Rating, Language and Country columns with no data and named it MovieReview.
18.	List all movies that have the same director but different genres, displaying the director's name, both movie titles, and their respective genres.
19.	Retrieve the title, director, and box office earnings for all movies that were released after 2010, along with their ratings.
20.	List all directors and the number of movies they have directed, but only include directors who have directed more than 1 movie.

Test Paper- D

Table Schema

Books Table:

- **book_id:** stores the unique ID for each book.
- **title:** stores the title of the book.
- **author_id:** stores the ID of the author who wrote the book (references the authors table).
- **genre:** stores the genre of the book (e.g., Fiction, Non-fiction, etc.).
- **publish_year:** stores the year the book was published.
- **available_copies:** stores the number of copies of the book currently available in the library.

Authors Table:

- **author_id:** stores the unique ID for each author.
- **name:** stores the full name of the author.
- **birth_year:** stores the year the author was born.
- **nationality:** stores the nationality of the author.

Members Table:

- **member_id:** stores the unique ID for each member of the library.
- **name:** stores the full name of the member.
- **email:** stores the email address of the member
- **phone_number:** stores the contact number of the member.
- **membership_date:** stores the date when the member joined the library.

Loans Table:

- **loan_id:** stores the unique ID for each loan.
- **book_id:** stores the ID of the book that is being loaned (references the books table).
- **member_id:** stores the ID of the member borrowing the book (references the members table).
- **loan_date:** stores the date when the book was borrowed.
- **return_date:** stores the date by which the book must be returned.

Consider above table schema and write following queries:

1.	Display the top 3 percentages books order by title in descending.
2.	Display a distinct list of genres.
3.	Insert a new book into the books table. ('The Adventures of Sherlock Holmes', 2, 'Mystery', 1892, 5)
4.	Update the number of available copies is 10 for a book whose book_id is available.
5.	Delete a member from the members table whose member_id is 4.
6.	Add a new column language varchar(20) to the books table.
7.	Truncate all data from the loans table. (Using Truncate)
8.	Find books whose title starts with ‘H’ and end with ‘L’.
9.	Find authors whose name does not ends with vowel.
10.	Find Lenth of ‘Manish Pandey’

11.	Calculate your age in year.
12.	Display the total number of books by genre.
13.	Display the minimum, maximum, and average number of available copies for each genre whose book_id is available.
14.	Display the title of books where the author was born before 1970.(Using Sub query)
15.	Create a view View_Member whose membership date is not available from members table
16.	Find the title of books that have been borrowed the most (the top 1 book) and the corresponding author name (Using sub Query)
17.	Display the loan_id ,member name ,title ,loan date whose member name is 'Raj'.
18.	List the titles of books that have been borrowed by members who registered before 2020.(using Sub query)
19.	Display the total number of books borrowed by each member.
20.	Display the title of books that have not been borrowed by any members.

Test Paper- E

Table Schema

Table-1: Restaurants:-

- **RestaurantID**: stores the unique ID for each restaurant.
- **RestaurantName**: stores the name of the restaurant.
- **Cuisine**: stores the type of cuisine the restaurant serves (e.g., French, Japanese, Italian).
- **Rating**: stores the average customer rating for the restaurant (out of 5).

Table-2: Customers:-

- **CustomerID**: stores the unique ID for each customer.
- **CustomerName**: stores the full name of the customer.
- **City**: stores the city where the customer resides.
- **Membership**: stores the membership level of the customer (e.g., Gold, Silver, Bronze).

Table-3: Orders:-

- **OrderID**: stores the unique ID for each order.
- **CustomerID**: stores the ID of the customer who placed the order (references the Customers table).
- **RestaurantID**: stores the ID of the restaurant where the order was placed (references the Restaurants table).
- **OrderDate**: stores the date and time when the order was placed.
- **TotalAmount**: stores the total amount of the order.

Table-4: MenuItems:-

- **MenuItemID**: stores the unique ID for each menu item.
- **RestaurantID**: stores the ID of the restaurant that offers the menu item (references the Restaurants table).
- **ItemName**: stores the name of the menu item.
- **Price**: stores the price of the menu item.
- **Category**: stores the category of the menu item (e.g., Main Course, Appetizer, Sushi, Soup).

Consider above table schema and write following queries:

1.	Display unique city of customers who have 'gold' membership.
2.	Display top 2 rating with restaurant names.
3.	Insert new restaurant in restaurant table. (5, 'Burger Junction', 'American', 4.1)
4.	Update customer id to 4 in orders table where amount is 60.
5.	Remove the costumer who belongs to Chicago city.
6.	Change column name Total Amount to Amount in Orders table.
7.	Delete Menu Items table.
8.	Display name and city of those customers whose membership contains 4 letters.
9.	Display 3rd to 7th character of restaurant name from restaurants table.
10.	Write a query to subtract 1 year from current date.
11.	Find max amount of all orders.

12.	Display city with the total number of customers.
13.	display restaurant names with average rating greater than 4.5.
14.	Find the highest-rated restaurant and its details.
15.	Subquery to get restaurants that are visited by customers from 'New York'.
16.	Create a View to list customers and their cities.
17.	Get all customers and their orders (including customers without order)
18.	Generate a combination of every customer with every restaurant.
19.	List all restaurants and the corresponding orders.
20.	Get the total amount spent by each customer at each restaurant, along with the customer and restaurant names. Include customers who have not ordered from certain restaurants, showing NULL for those cases.